

A Brutally Honest Evaluation of Multi-Tiered Semantic and Refactoring-Aware Merge Architectures

Introduction: The Crisis of Concurrent Software Evolution

The integration of concurrent software modifications represents the most foundational, yet persistently fragile, pillar of modern collaborative software engineering. The ubiquitous reliance on unstructured, text-based merging algorithms—primarily the forty-year-old diff3 algorithm embedded within version control systems like Git, Subversion, and Mercurial—has created a systemic vulnerability in continuous integration and continuous deployment (CI/CD) pipelines.¹ Unstructured merge tools treat source code as flat sequences of text lines, remaining entirely agnostic to the underlying syntactic structures and semantic behaviors of programming languages. Consequently, when developers execute structural transformations such as refactorings—renaming methods, extracting classes, or moving fields—concurrently with independent edits to the same logical blocks, textual merging tools inevitably fail.¹ This failure manifests in two highly disruptive forms. The first is the generation of false positive conflicts, where the merging tool detects overlapping textual edits but fails to recognize that the changes are semantically compatible and can be automatically resolved.¹ The second, and exponentially more dangerous, is the generation of false negative conflicts, often termed semantic conflicts. In these scenarios, textual edits do not physically overlap, prompting the tool to silently execute a successful text merge that subsequently introduces compilation failures, broken tests, or insidious runtime regressions, such as accidental method overrides or stale reads.² Refactoring operations are implicated in up to 22% of all merge conflicts and tend to produce substantially larger and more complex conflict blocks, drastically increasing the cognitive load required for manual resolution.¹

To address these vulnerabilities, researchers have proposed various structural, semi-structured, and refactoring-aware merging techniques. The architecture under evaluation presents a highly ambitious, six-stage pipeline designed to systematically eliminate both syntactic and semantic merge conflicts. It attempts this by layering Abstract Syntax Tree (AST) differencing, graph-based structural merging, refactoring detection, Code Property Graph (CPG) dataflow analysis, and Satisfiability Modulo Theories (SMT) formal verification.¹ While the theoretical underpinnings of this pipeline represent a comprehensive survey of academic software analysis techniques, an exhaustive operational deconstruction reveals profound redundancies, computational bottlenecks, and paradigm collisions. This report provides a direct, brutally honest analysis of the individual tools comprising this proposed architecture, evaluates the empirical efficacy of the paradigms they represent, and critically

assesses whether this specific architectural composition has a viable place in modern, large-scale, AI-accelerated software development.

The Impact of Generative AI on Code Volatility and Merge Architectures

Before dissecting the specific algorithmic mechanics of the proposed pipeline, it is mandatory to contextualize the environment in which a modern merge driver operates. The years 2024 through 2026 have witnessed a paradigm shift in software engineering, characterized by the hyper-accelerated enterprise adoption of generative AI code assistants and autonomous programming agents.⁴ By 2025, enterprise adoption of AI-powered code assistants reached 90%, with models actively writing or modifying approximately 41% of all enterprise code.⁵ This shift has profound implications for merge architectures. Large Language Models (LLMs) like Claude 3.5 Sonnet, GPT-4o, and specialized tools like Amazon CodeWhisperer and Augment Code are no longer utilized merely for single-line autocompletion; they are deployed for repository-scale legacy modernization and multi-file refactoring.⁴ These AI agents can analyze outdated logic, rewrite UI monoliths into cloud-native microservices, and optimize entire class hierarchies in a matter of seconds.⁷

Consequently, the volume and velocity of structural code changes have skyrocketed. Each AI-generated feature request or modernization sprint triggers massive diffs spanning thousands of interdependent files, leading to what industry practitioners describe as "merge conflict avalanches".⁸ Furthermore, while LLMs excel at generating syntactic code, they are highly prone to hallucination.⁸ They frequently hallucinate imports, miss subtle domain-specific semantic constraints, and generate code that appears textually and syntactically flawless but violates the broader semantic contracts of the repository.⁸

This AI-driven volatility establishes an absolute, non-negotiable mandate for merge drivers that can comprehend syntax and semantics. Textual merge drivers are fundamentally incapable of validating the massive structural shifts generated by autonomous agents. However, this same environment dictates that any semantic merge driver must be exceptionally fast. CI/CD pipeline performance metrics—specifically "Merge to Deploy Time"—are critical to team velocity.¹¹ If a refactoring-aware merge architecture introduces severe latency, it will cause the CI/CD pipeline to collapse under the sheer volume of AI-generated commits. The proposed architecture must be evaluated not merely on its theoretical ability to resolve conflicts, but on its ability to do so within the unforgiving temporal constraints of an AI-augmented DevOps pipeline.

The Proposed Architecture: A Theoretical Deconstruction

The proposed semantic merge driver architecture is designed as a pluggable, multi-layered system orchestrated by a Python driver, though the underlying tooling is almost exclusively locked into the Java ecosystem.¹ The architecture constructs a sequential gauntlet of analysis

and resolution engines, progressing from simple textual merging to formal mathematical verification.¹

To ascertain the viability of this pipeline, we must dissect each of its six discrete stages, exposing the algorithmic mechanics, the theoretical justifications, and the operational flaws of the selected tools.

The Textual Baseline and the Illusion of Code Property Graphs (Step 0)

The pipeline initiates at Step 0 by executing a standard textual three-way merge utilizing git merge-file.¹ Recognizing that this unstructured approach will blindly merge semantically incompatible code, the architecture immediately introduces Joern to perform static analysis on the resulting text-merged file.¹ Joern is an open-source code analysis platform originally designed for vulnerability discovery, which operates by parsing source code into a Code Property Graph (CPG).¹³

A Code Property Graph is a massive, interconnected data structure that unifies the Abstract Syntax Tree (AST), the Control Flow Graph (CFG), and the Program Dependence Graph (PDG) into a single, queryable graph schema.¹³ Within this architecture, Joern is tasked with running Scala-based Domain Specific Language (DSL) queries to detect elementary logical flaws, such as infinite loops or invalid loop bounds introduced by the textual merge.¹

Deploying Joern at this stage is an architectural miscalculation. Textual merges frequently produce syntactically invalid code, characterized by broken brackets, duplicated modifiers, or severed method signatures.¹ Joern, while utilizing fuzzy parsing techniques like island grammars to handle incomplete code snippets, fundamentally relies on a coherent syntactic structure to accurately generate the CFG and PDG layers of the Code Property Graph.¹⁵

Feeding a broken, text-merged file into a CPG generator will either cause the parser to fail entirely or result in a disjointed, fragmented graph that triggers continuous false positive alerts in the downstream Scala queries. Furthermore, the performance overhead of generating a full CPG for large enterprise repositories is immense; recent research indicates that analyzing execution paths through CPG traversal requires complex slicing techniques to reduce code size by up to 90% just to make the analysis computationally tractable.¹⁰ Utilizing CPG generation as a baseline, synchronous validation step for every git merge introduces catastrophic latency for zero tangible benefit.

Abstract Syntax Tree Differencing: The GumTree Evolution (Step 1)

Step 1 attempts to elevate the pipeline from unstructured text to structural analysis by introducing GumTree to generate AST edit scripts.¹ By comparing the base version against the left and right branches, the driver extracts modified AST node paths to detect overlaps, flagging potential semantic conflicts such as atomic updates or scope capture issues.¹ To understand the viability of this step, one must trace the algorithmic evolution of GumTree.

The original 2014 GumTree algorithm executed AST differencing through a bipartite process. The top-down phase utilized a greedy algorithm to identify the largest isomorphic subtrees between the source and destination ASTs.¹ Following this, a bottom-up phase propagated mappings to parent nodes based on a dice coefficient, which measured the ratio of common matched descendants.¹

The critical vulnerability of the 2014 architecture resided in its third phase: the recovery phase. To capture unmatched nodes that were skipped during the top-down and bottom-up traversals, GumTree deployed an optimal tree-edit distance algorithm.¹ This algorithm computed the absolute minimum sequence of edits required to transform one subtree into another, but it possessed a brutal $O(n^3)$ time complexity, rendering it mathematically incapable of processing large Abstract Syntax Trees.¹

To prevent the algorithm from hanging indefinitely, the authors artificially constrained the recovery phase with a hyperparameter known as the `max_size` threshold.¹ If the number of descendants in a subtree exceeded this threshold (defaulted to 1000 nodes), the optimal recovery algorithm aborted entirely.¹ The application of this threshold forced a detrimental operational trade-off. A low threshold accelerated execution but forfeited vital mappings, resulting in noisy, inaccurate edit scripts filled with spurious insert and delete actions rather than precise updates.¹ Conversely, a high threshold paralyzed the system; empirical tests on the Defects4J dataset demonstrated that increasing the `max_size` threshold from 1000 to 1500 doubled the execution time while providing minimal reductions in edit script size.¹ For automated pipelines consuming these edit scripts, an aborted recovery phase meant that unchanged elements were erroneously flagged as deleted and re-inserted, confusing downstream conflict detection algorithms.¹

Fortunately, the 2024 iteration of GumTree decisively resolves this bottleneck by abandoning the optimal $O(n^3)$ recovery phase in favor of a hybridized simple heuristic.¹ The modernized recovery algorithm employs a longest common subsequence matching technique.¹ It first executes strict isomorphism checks on unmatched children. Crucially, it follows this by performing structural isomorphism checks that deliberately ignore leaf-node labels.¹ This allows the algorithm to seamlessly detect updated identifiers and variables without triggering full tree-edit distance computations.¹ Finally, it seeks pairs of nodes whose types appear uniquely in both children's lists, mapping them iteratively.¹

This architectural pivot entirely eliminates the `max_size` threshold constraint.¹ Extensive empirical validations demonstrate that the 2024 simple heuristic algorithm achieves a massive total speedup of the matching time between 50x and 281x compared to the 2014 default, while simultaneously generating edit scripts with a median size 50% smaller.¹ In a qualitative analysis of 100 file pairs, the edit scripts produced by the new simple heuristic were rated as vastly superior to the optimal recovery by both experts and external participants, primarily because it completely eliminated the spurious insert/delete actions caused by aborted recoveries on large class definitions.¹

Despite these monumental advancements in execution speed, deploying GumTree in Step 1 remains problematic due to its inherent language-agnostic ambitions. By attempting to serve

as a universal structural parser, GumTree fundamentally ignores language-specific semantic clues.¹ It enforces a strict one-to-one mapping constraint, rendering it incapable of comprehending multi-mappings—such as a developer duplicating a code fragment to reuse it, or consolidating duplicated conditional fragments into a unified block.¹ Furthermore, GumTree's algorithmic blindness to semantic roles frequently leads to semantically incompatible mappings. For example, it might erroneously map a type reference to a variable identifier simply because both are parsed as generic SimpleName nodes by the underlying parser, corrupting the edit script and polluting the pipeline's conflict detection logic.¹

Structured Merge Backends: The JDime and Spork Monoculture (Step 2)

Step 2 of the proposed architecture replaces the textual merge backend with structured AST-level merge tools, specifically deploying either JDime or Spork.¹ The objective is to produce a structurally coherent merged file, eliminating the formatting and ordering conflicts that confound text-based tools, thereby reducing noise for the subsequent semantic analysis layers.¹

Both JDime and Spork represent valid academic explorations into semi-structured and structured merging. JDime functions as a syntactic merge tool that supports line-based, semi-structured, and structured modes, utilizing an extensible compiler to handle abstract syntax trees.¹ Spork operates similarly, attempting to resolve ASTs and merge them based on structural hierarchies rather than raw text.¹

The fatal flaw of Step 2 is not the concept of structured merging, but the devastating operational constraint of the chosen tools: both JDime and Spork are strictly and exclusively engineered for the Java programming language.¹ Spork relies on the Spoon library for AST representation, which is entirely Java-dependent.¹ JDime's repositories and documentation confirm a 100% Java codebase with no abstraction layers capable of ingesting syntax trees from other languages.¹

In the contemporary software engineering landscape of 2025 and 2026, polyglot development is the default paradigm. Enterprise data pipelines are aggressively transitioning to Rust for memory efficiency and safety¹⁸; backend microservices seamlessly blend Go, Python, and C#²⁰; and frontend architectures are universally dominated by TypeScript. An architectural pipeline that can only structurally merge Java files is an archaic relic, utterly useless to a DevOps team orchestrating a modern, multi-language repository.²¹ Furthermore, relying on highly specialized academic tools like JDime and Spork introduces severe maintenance risks, as these tools lack the robust community support and rapid update cycles required to keep pace with evolving language specifications.

High-Precision Refactoring Detection: The RefactoringMiner Paradigm (Step 3)

Recognizing that basic AST differencing and structured merging cannot comprehend the semantic intent of code movement, Step 3 introduces RefactoringMiner 2.0 as a pre-merge hook.¹ The goal is to detect semantic transformations—such as method renames, extractions, and inlines—with high precision, passing this intelligence to downstream routers to handle dangling references and scope capture conflicts.¹

Refactoring detection is a notoriously difficult computational problem because refactoring operations are rarely performed in isolation; they are frequently entangled with feature additions, bug fixes, or sequential refactoring operations, a phenomenon known as refactoring masking.¹ Historically, detection tools like RefDiff and Ref-Finder attempted to solve this by relying on arbitrary code similarity thresholds, utilizing techniques like TF-IDF weighting or Cosine similarity to compare code fragments across commits.¹

This threshold dependency is a fatal flaw in automated CI/CD pipelines. A universal threshold cannot mathematically satisfy the architectural variances across diverse codebases. Different application domains, architectural styles, and framework paradigms dictate different code modification patterns; a threshold optimized for a Spring Boot monolith will inevitably trigger either an avalanche of false positives or a drought of false negatives when applied to an Android mobile application.¹

RefactoringMiner 2.0 represents a breakthrough because it bypasses the similarity threshold trap entirely. It optimizes its search space by processing Git revisions strictly at the commit level, analyzing only added, deleted, and changed files.¹ The algorithm first matches code elements in a top-down fashion based on identical signatures. For the remaining unmatched entities, it deploys a sophisticated, multi-round statement-level matching algorithm.¹

Crucially, to handle statements that have been obfuscated by refactoring mechanics, RefactoringMiner 2.0 applies two advanced pre-processing techniques: abstraction and argumentization.¹ Abstraction strips away structural wrappers—reducing a return expression; or an if (expression) to just the core expression—allowing statements with different AST node types to be compared directly.¹ Argumentization handles the complex mechanics of method extraction by replacing parameter names in the extracted method with the actual arguments passed at the call site in the original method.¹

Following this pre-processing, the algorithm executes syntax-aware AST node replacements, finding the minimum edit distance to make two statements textually identical.¹ It ensures that only compatible AST nodes are replaced (e.g., types with types, operators with operators) without ever reverting to an arbitrary similarity percentage.¹ This methodology allows RefactoringMiner 2.0 to detect 40 distinct refactoring types with an unprecedented 99.6% precision and 94% recall across a massive oracle of 7,226 true refactoring instances.¹

However, the architecture specifies the use of RefactoringMiner 2.0, which is currently obsolete. The subsequent release, RefactoringMiner 3.0 (also known as RM-ASTDiff), dramatically alters the landscape by unifying refactoring detection directly with AST differencing.¹ RefactoringMiner 3.0 expands support to over 100 refactoring types and breaks the strict one-to-one mapping constraint enforced by tools like GumTree, natively supporting one-to-many and many-to-one multi-mappings.¹ By leveraging language-specific clues—such

as utilizing a method call injection to precisely locate where an extracted method's body resides—it achieves flawless 100% precision in inter-file mapping and completely eliminates the semantically incompatible mappings that plague generic structural tools.¹ Furthermore, RefactoringMiner has expanded beyond the Java monoculture, natively supporting Python and Kotlin by integrating specialized parser infrastructures.¹

The existence of RefactoringMiner 3.0 immediately invalidates the separation of Step 1 (GumTree) and Step 3 (RefactoringMiner 2.0) in the proposed architecture. Running GumTree for AST differencing and then subsequently running an older version of RefactoringMiner for detection is computationally wasteful and logically incoherent when a single, unified tool provides vastly superior, refactoring-aware AST differencing.

The Routing Collision: Graph-Based vs. Operation-Based Merging (Step 4)

Step 4 acts as the central nervous system of the proposed pipeline. It introduces a dynamic "Merge Router" that dictates the flow of execution based on the findings of Step 3.¹ If refactorings are detected, the pipeline abandons the structured merge tools (JDime/Spork) and routes the merge to IntelliMerge, a graph-based refactoring-aware merge tool.¹ Following the merge, Joern is invoked again to perform deep dataflow and taint analysis on the resulting CPG.¹

This stage forces a critical evaluation of the two dominant paradigms in refactoring-aware merging: graph-based merging (represented by the proposed IntelliMerge) and operation-based merging (represented by tools like RefMerge and MolhadoRef).

The Fallacy of Graph-Based Merging (IntelliMerge)

IntelliMerge operates on the premise that converting source code into a formal mathematical structure allows for perfect semantic alignment. It parses the base, left, and right Java source files and translates them into Program Element Graphs (PEGs).¹ In these directed, weighted graphs, nodes represent program elements (classes, methods, fields) and edges represent semantic relationships (method calls, field accesses, inheritance).¹

The merging algorithm executes a top-down phase to match nodes by signature, moving from the root to the leaves.¹ This is followed by a bottom-up phase designed to match refactored elements. To do this, IntelliMerge calculates the similarity between unmatched nodes using a combination of context similarity (evaluating the similarity of incoming and outgoing edges) and body-AST similarity.¹ It attempts to handle 1-to-1 matches (like renaming or moving a method) by ranking candidates based on context, body similarity, and location distance.¹ For m-to-n matches (like extracting a method from multiple origins or inlining a method to multiple targets), it extends the context of the candidates and ranks them by the updated context similarity.¹ Following the matching phase, the graphs are merged and serialized back into Java source code.¹

While theoretically elegant, massive empirical evaluations expose severe, insurmountable practical limitations in the graph-based paradigm. In a rigorous study of 2,001 merge scenarios

characterized by refactoring-related conflicts across 20 open-source projects, IntelliMerge successfully and completely resolved only 78 scenarios (4%).¹

More alarmingly, while it managed to reduce the total conflicting Lines of Code (LOC) in roughly 20% of scenarios (primarily by resolving simple ordering and formatting conflicts), it actively worsened the developer experience in a massive number of cases. IntelliMerge significantly increased the conflicting LOC in 597 scenarios (30%), generating a median increase in conflict size of 169%.¹ In certain projects, such as jackson-core, it increased conflict blocks by over 4200% compared to standard Git.¹

Metric	Git (Textual)	IntelliMerge (Graph)	RefMerge (Operation)
Completely Resolved Scenarios	Baseline	78 (4%)	143 (7%)
Scenarios with Reduced Conflicting LOC	N/A	408 (20%)	372 (19%)
Scenarios with Increased Conflicting LOC	N/A	597 (30%)	214 (11%)
False Positives (50-scenario qualitative sample)	243	707	188
False Negatives (50-scenario qualitative sample)	5	71	0

Table 1: Comparative empirical data derived from 2,001 merge scenarios across 20 open-source projects.¹

The root cause of IntelliMerge's failure at scale is twofold. First, its reliance on context similarity inherently involves falling back on the very similarity score thresholds that RefactoringMiner proved were flawed.¹ In scenarios characterized by a high volume of refactorings or extensive file modifications (e.g., class-level extractions), the graph structures diverge too radically for the similarity thresholds to reliably identify matches, leading to catastrophic node matching errors.¹

Second, IntelliMerge evaluates graphs holistically but fails to account for how specific refactoring operations on disparate branches logically interact with one another. This algorithmic blindness results in a staggering 707 false positive conflict blocks in a 50-scenario qualitative sample—a 192% increase over standard Git.¹ Furthermore, it introduced 71 false negatives (where conflicts should have been reported but were silently merged), representing a 1,320% increase in dangerous semantic conflicts compared to the baseline.¹ Routing merges to IntelliMerge in Step 4 guarantees that 30% of complex merges will be catastrophically worsened.

The Triumph of Operation-Based Merging (RefMerge)

The operation-based paradigm, pioneered by MolhadoRef and modernized for native Git environments by RefMerge, takes a fundamentally different and empirically superior approach. It treats refactorings not as structural anomalies to be mapped via graph similarity, but as discrete mathematical operations that can be inverted and replayed.¹

RefMerge operates in a precise five-step sequence. First, it utilizes RefactoringMiner to detect refactorings on the left and right branches independently, ensuring precise detection without reliance on arbitrary similarity thresholds.¹ It then systematically simplifies these operations, collapsing transitive refactoring chains. For example, if method A is renamed to B, and in a subsequent commit B is renamed to C, RefMerge collapses this into a single operation: A renamed to C.¹ It also tracks refactoring chains on the same element, ensuring that a method rename is correctly tracked even if its parent class is subsequently renamed.¹

Following simplification, the tool inverts the refactorings in a top-down order, transforming the codebase back into a "refactoring-free" state.¹ At this juncture, a standard textual Git merge is executed on the purified code. Because the structural shifts have been temporarily removed, textual merge tools excel at resolving the remaining logic edits.¹

Finally, RefMerge analyzes the refactorings from both branches using a rigorous conflict matrix. It checks for commutative logic (refactorings that can be applied in any order without interference) or strict semantic conflicts.¹ A prime example of a semantic conflict detected by this matrix is an accidental override or overload: if Branch 1 renames Method X to Z, and Branch 2 extracts a new Method Z within the same class hierarchy, RefMerge flags this as a critical conflict.¹ Textual and graph-based tools blindly merge this, causing catastrophic runtime failures. If no conflicts are detected, RefMerge replays the valid refactorings sequentially (in a bottom-up order) onto the merged codebase, updating all references appropriately.¹

The empirical superiority of the operation-based approach is undeniable. In the same 2,001-scenario evaluation, RefMerge completely resolved 143 scenarios (7%)—nearly double the success rate of IntelliMerge.¹ Crucially, RefMerge is highly resilient to the density of refactorings within a scenario. It reduced false positives by 23% compared to Git, entirely eliminated false negatives, and increased conflicting LOC in only 11% of scenarios (with a median increase of just 14%, compared to IntelliMerge's 169%).¹

Furthermore, RefMerge shatters the previous academic assumption that operation-based merging cannot handle complex refactorings. The creators of IntelliMerge theorized that operations like Extract Method or Inline Method could not be inverted.¹ RefMerge proves this false; an Extract Method is perfectly inverted by executing an Inline Method operation on the extracted code block, allowing the textual merge to process the original unextracted logic, before replaying the extraction on the newly merged code.¹

The inclusion of IntelliMerge in the proposed architecture is a critical error. The data unequivocally dictates that an operation-based approach utilizing RefactoringMiner's threshold-less detection yields drastically fewer false positives, eliminates threshold-induced graph matching failures, completely eliminates false negatives, and provides a demonstrably

safer overall merge resolution.

The Pipeline Latency Crisis: Why Semantic Verification Fails in CI/CD

If the routing logic in Step 4 is flawed, the subsequent integration of Joern for dataflow analysis and the deployment of SafeMerge for formal verification in Step 5 represent a profound disconnect between academic theory and the brutal realities of industrial software engineering.

The Overhead of Code Property Graphs (Joern)

In the proposed architecture, after the merge is completed, Joern is invoked to perform dataflow and taint analysis. The intent is to detect "stale reads" (e.g., Branch A clears a list while Branch B attempts to iterate over it) or invalid loop bounds introduced by the merge.¹

While Joern is a formidable tool for asynchronous vulnerability discovery and security auditing²⁴, weaponizing it as a synchronous, blocking mechanism within a merge driver pipeline is computationally suicidal. Constructing a complete Code Property Graph for a large enterprise repository requires immense parsing overhead, as it must compute the AST, CFG, and PDG for thousands of interdependent files.¹³

In modern DevOps, "Merge to Deploy Time" and "Average Pipeline Run Time" are critical metrics governing team velocity.¹¹ Introducing heavyweight static analysis into the actual git merge driver hook guarantees unacceptable pipeline degradation. Empirical studies on code instrumentation and analysis in CI pipelines demonstrate that introducing heavy monitoring tools can decrease overall throughput by up to 30% and trigger erratic timeout behaviors.¹² The recent development of LLMxCPG—which utilizes LLMs to slice and reduce code size by 67% to 90% prior to Joern analysis just to make the processing time viable—confirms that native CPG generation is too bloated for rapid, real-time evaluation without extreme, context-aware optimization.¹⁰ Running Joern synchronously on every merge will inevitably lead to pipeline timeouts and developer revolt.

The Formal Verification Bottleneck (SafeMerge and Z3)

Step 5 proposes using SafeMerge, a tool leveraging the Microsoft Z3 SMT solver, as the final verification gate. The objective is to mathematically prove semantic conflict-freedom.¹

SafeMerge operates by computing a 4-way AST diff (incorporating the base, left, right, and merged programs) to represent the codebase as a shared program with "holes" indicating the modifications.²⁷ It partitions the program into segments devoid of holes, replacing complex logic with uninterpreted functions. It then attempts to generate relational invariants—utilizing Houdini-style loop invariant generation—to mathematically prove that the merged program does not introduce any unwanted, divergent behaviors that were not present in the parent branches.²⁷

If Joern introduces severe latency, SafeMerge introduces absolute paralysis. While the Z3 theorem prover is a marvel of logical computation used extensively in hardware verification and

critical systems²⁹, formal verification suffers from catastrophic state-space explosion when applied to complex, general-purpose software systems.²

The literature surrounding SafeMerge explicitly acknowledges that generating inductive loop invariants and performing precise relational reasoning scales poorly with the absolute size of the program.² While the authors implemented differential program verification to scale the verification to the size of the *edits* rather than the whole program, the process is still extraordinarily heavy. It was evaluated successfully on exactly 52 carefully selected real-world merge scenarios.²⁷ The idea of deploying Satisfiability Modulo Theories solving automatically on every single developer pull request in a large-scale repository is entirely divorced from industry reality.

As noted by industry practitioners, while formal methods provide unparalleled mathematical guarantees, they are practically reserved for hyper-critical, isolated components (e.g., cryptographic modules, automotive braking systems, or drone autopilots) due to the immense cost of execution time and specification definition.³¹ Mandating SMT-based formal verification as a blanket requirement for general Git merges is architectural over-engineering of the highest order. It will stall pull requests, fail CI runs due to timeouts, and destroy development velocity.²

The Polyglot Reality: Mergiraf and the Tree-Sitter Revolution

Perhaps the most damning critique of the proposed architecture is its absolute, unwavering subservience to the Java ecosystem. The pipeline orchestration relies sequentially on JDime, Spork, IntelliMerge, RefactoringMiner, and SafeMerge.¹ With the exception of Joern (which supports multiple languages but is utilized here via Scala queries specifically for Java), every structural, semantic, and verification tool in this pipeline is strictly engineered to parse, analyze, and merge Java source code.¹

The year is 2026. Modern software development is intrinsically polyglot. Enterprise data pipelines are aggressively transitioning to Rust for memory efficiency and safety, achieving massive cost reductions.¹⁸ Backend microservices seamlessly blend Go, Python, and C#.²⁰ Frontend architectures are universally dominated by TypeScript and JavaScript. An architectural pipeline that can only structurally merge Java files is an archaic relic, utterly useless to a DevOps team orchestrating a modern, multi-language repository.²¹

The industry has already recognized the futility of building monolithic, language-specific AST tools. The shift is decisively moving toward unified, high-performance parsing frameworks, most notably tree-sitter. This shift is perfectly encapsulated by **Mergiraf**, a production-ready, syntax-aware Git merge driver written in Rust.¹

Mergiraf explicitly solves the polyglot problem by utilizing tree-sitter grammars to parse and understand the syntax trees of over 33 different programming languages and data formats.¹ It operates on a hybrid, pragmatically optimized methodology: it defaults to lightning-fast line-based merging. Only when a conflict occurs does it engage its tree-aware matching

engine to reconcile the divergent structures.¹

Crucially, Mergiraf requires no complex threshold tuning or SMT solvers. It utilizes a completely declarative approach to language definition, making it trivial to add support for new languages.¹ It understands language-specific semantics regarding commutativity; for instance, it knows that the order of Java import statements, HTML attributes, or JSON object keys does not matter, allowing it to seamlessly resolve ordering conflicts that would break standard Git.¹ Furthermore, it employs strict safety heuristics; if the syntax-aware merging appears too optimistic or encounters a suspicious structural anomaly, it immediately falls back to inserting conflict markers, prioritizing safety over automated guesswork.¹

While Mergiraf currently lacks the deep, commit-level refactoring-awareness of RefactoringMiner, its architecture aligns flawlessly with the performance, safety, and multi-language requirements of modern CI/CD pipelines.³⁵ It is designed to be fast enough for interactive developer use, directly replacing the default Git merge driver without requiring cloud infrastructure or heavy JVM spin-ups.¹

Conclusion and Architectural Verdict

To address the core query of this analysis: *Does this architecture have a place to be?*

In an academic laboratory researching the theoretical limits of program analysis and formal verification, yes. As a proposed production system for actual software engineering teams operating in the high-velocity, AI-augmented landscape of 2026, **absolutely not.**

The proposed architecture is an academic Frankenstein—a concatenation of computationally heavy, overlapping systems that solve the same problems redundantly while introducing catastrophic performance overheads and language lock-in.

1. **Massive Redundancy:** Step 1 (GumTree), Step 2 (JDime/Spork), and Step 3 (RefactoringMiner 2.0) are completely cannibalized by the existence of RefactoringMiner 3.0. There is no logical justification for generating standard AST edit scripts, performing a structured AST merge, and *then* running a separate refactoring detection pass. RefactoringMiner 3.0 unifies AST differencing and refactoring detection with near-perfect precision, natively handling the move operations and multi-mappings that JDime and GumTree stumble over, while expanding support to Python and Kotlin.¹
2. **Algorithmic Inferiority in Step 4:** Routing merges to IntelliMerge relies on a flawed graph-based paradigm. Empirical evidence unequivocally proves that IntelliMerge's reliance on similarity thresholds actively increases conflicting lines of code in 30% of scenarios, introducing massive false positive and false negative rates.¹ The operation-based approach (RefMerge), fueled by the threshold-less detection of RefactoringMiner, is mathematically and practically superior.¹
3. **The CI/CD Death Sentence in Step 5:** Injecting Joern CPG traversal and SafeMerge Z3 SMT verification into a synchronous Git merge hook will obliterate developer velocity. The latency introduced by generating property graphs and attempting to mathematically prove relational invariants across 4-way product programs cannot be justified outside of highly specialized, mission-critical environments.²

4. **The Java Monoculture:** A pipeline that cannot merge a Python microservice, a Rust data processor, or a TypeScript frontend has zero viability as a unified enterprise solution.¹

If the goal is to build a practical, high-performance, refactoring-aware merge system that actually survives contact with an enterprise engineering team, the architecture must be violently pruned and modernized.

The baseline textual merge should be replaced immediately with a tree-sitter-backed engine like Mergiraf, providing instant, high-speed structural merging capabilities across dozens of languages.¹ For languages where deep refactoring detection is available (such as Java and Python via RefactoringMiner 3.0), the architecture should utilize an operation-based "invert, merge, replay" pipeline similar to RefMerge.¹ This eliminates the threshold-induced graph matching failures of IntelliMerge. Finally, heavy semantic verification tools like Joern and SafeMerge must be completely excised from the synchronous merge driver. They belong in a dedicated CI/CD security and compliance pipeline that runs asynchronously *after* a pull request has been successfully merged.

By dismantling this bloated pipeline and reconstructing it around the speed of tree-sitter parsing, the precision of operation-based merging, and the proper asynchronous decoupling of heavy semantic verification, the industry can achieve the promise of refactoring-aware merging without sacrificing the velocity required by modern software development.

Works cited

1. ARCHITECTURE.md
2. Safe program merges at scale: A grand challenge for program repair research - Microsoft, accessed March 26, 2026, <https://www.microsoft.com/en-us/research/blog/safe-program-merges-at-scale-a-grand-challenge-for-program-repair-research/>
3. How code composition strategies affect merge conflict resolution? | Journal of Software Engineering Research and Development - SBC, accessed March 26, 2026, <https://journals-sol.sbc.org.br/index.php/jserd/article/view/3638>
4. AI Agentic Programming: A Survey of Techniques, Challenges, and Opportunities - arXiv, accessed March 26, 2026, <https://arxiv.org/html/2508.11126v1>
5. Enterprise AI Code Assistants for Air-Gapped Environments | IntuitionLabs, accessed March 26, 2026, <https://intuitionlabs.ai/articles/enterprise-ai-code-assistants-air-gapped-environments>
6. AI code generation: Best practices for enterprise adoption in 2025 - DX, accessed March 26, 2026, <https://getdx.com/blog/ai-code-enterprise-adoption/>
7. Unlocking the Future of Code Refactoring: Bridging AI and Large Scale Automation for Legacy Modernization. | by Siva Sreeraman | Medium, accessed March 26, 2026, <https://medium.com/@siva.sreeraman/unlocking-the-future-of-code-refactoring-bridging-ai-and-large-scale-automation-for-legacy-7e1535c5a9d2>
8. AI Code Refactoring: Tools, Tactics & Best Practices, accessed March 26, 2026, <https://www.augmentcode.com/tools/ai-code-refactoring-tools-tactics-and-best>

[-practices](#)

9. Through the Code Maze: AI vs Manual Refactoring - Devox Software, accessed March 26, 2026,
<https://devoxsoftware.com/blog/through-the-code-maze-ai-vs-manual-refactoring/>
10. LLMxCPG: Context-Aware Vulnerability Detection Through Code Property Graph-Guided Large Language Models - USENIX, accessed March 26, 2026,
<https://www.usenix.org/system/files/usenixsecurity25-lekssays.pdf>
11. CI/CD Pipeline Performance - minware, accessed March 26, 2026,
<https://www.minware.com/report/ci-cd-pipeline-performance>
12. Optimize code repository performance for better software delivery - Harness, accessed March 26, 2026,
<https://www.harness.io/harness-devops-academy/code-repository-performance-optimization>
13. Code Property Graph | Joern Documentation, accessed March 26, 2026,
<https://docs.joern.io/code-property-graph/>
14. joern.io - GitHub, accessed March 26, 2026, <https://github.com/joernio>
15. No pain no gain? Comparing 3 program analysis frameworks for C - Benjamin Steenhoek, accessed March 26, 2026,
<https://benjijang.com/posts/2022/03/no-pain-no-gain/>
16. LLMxCPG: Context-Aware Vulnerability Detection Through Code Property Graph-Guided Large Language Models - arXiv, accessed March 26, 2026,
<https://arxiv.org/html/2507.16585v1>
17. LLMxCPG: Context-Aware Vulnerability Detection Through Code Property Graph-Guided Large Language Models - USENIX, accessed March 26, 2026,
<https://www.usenix.org/system/files/usenixsecurity25-appendix-lekssays.pdf>
18. Rust Eats Python's, Java's Lunch in Data Engineering - The New Stack, accessed March 26, 2026,
<https://thenewstack.io/rust-eats-pythons-javas-lunch-in-data-engineering/>
19. Optimizing DevOps Pipelines for Rust Projects: Leveraging Cargo and CI/CD, accessed March 26, 2026,
https://dev.to/mark_mwendia_0298dd9c0aad/optimizing-devops-pipelines-for-rust-projects-leveraging-cargo-and-cicd-474d
20. ActRef: Enhancing the Understanding of Python Code Refactoring with Action-Based Analysis - arXiv, accessed March 26, 2026,
<https://arxiv.org/html/2505.06553v1>
21. Combine pipelines into one - azure devops - Stack Overflow, accessed March 26, 2026, <https://stackoverflow.com/questions/63838732/combine-pipelines-into-one>
22. pouryafard75/RM-ASTDiff - GitHub, accessed March 26, 2026,
<https://github.com/pouryafard75/RM-ASTDiff>
23. PyRef: Refactoring Detection in Python Projects - Morressier, accessed March 26, 2026,
<https://www.morressier.com/o/event/613a1f8b7f98360013889ed8/article/613b54401459512fce6a7cd8>
24. The Bug Hunter's Workbench | Impact - Joern, accessed March 26, 2026,

- <https://joern.io/impact/>
25. Code Property Graph Specification 1.1 - Joern, accessed March 26, 2026, <https://cpg.joern.io/>
 26. An empirical study on the performance overhead of code instrumentation in containerised microservices - ResearchGate, accessed March 26, 2026, https://www.researchgate.net/publication/393744178_An_empirical_study_on_the_performance_overhead_of_code_instrumentation_in_containerised_microservices
 27. Verified Three-Way Program Merge - Computer Science, accessed March 26, 2026, <https://www.cs.utexas.edu/~isil/verified-merge.pdf>
 28. SafeMerge: Verification of Conflict Freedom in Program Merges - GitHub, accessed March 26, 2026, <https://github.com/marcelosousa/safemerge>
 29. The Varied Forms of Verification with Z3 - Microsoft Research, accessed March 26, 2026, <https://www.microsoft.com/en-us/research/video/the-varied-forms-of-verification-with-z3/>
 30. The inner magic behind the Z3 theorem prover - Microsoft Research, accessed March 26, 2026, <https://www.microsoft.com/en-us/research/blog/the-inner-magic-behind-the-z3-theorem-prover/>
 31. Git-for-Data Semantics: Safe Branching & Merging at Scale - Bauplan Labs, accessed March 26, 2026, <https://www.bauplanlabs.com/post/git-for-data-formal-semantics-of-branching-merging-and-rollback-part-1>
 32. [1802.06551] Verifying Semantic Conflict-Freedom in Three-Way Program Merges - arXiv, accessed March 26, 2026, <https://arxiv.org/abs/1802.06551>
 33. Formal Verification of Code Conversion: A Comprehensive Survey - MDPI, accessed March 26, 2026, <https://www.mdpi.com/2227-7080/12/12/244>
 34. What's the verdict on Mergiraf? : r/git - Reddit, accessed March 26, 2026, https://www.reddit.com/r/git/comments/1rrrf0s/whats_the_verdict_on_mergiraf/
 35. Mergiraf | Technology Radar - Thoughtworks, accessed March 26, 2026, <https://www.thoughtworks.com/radar/tools/mergiraf>
 36. mergiraf: a driver for `git merge` that uses an understanding of programming language syntax to resolve merge conflicts : r/rust - Reddit, accessed March 26, 2026, https://www.reddit.com/r/rust/comments/1gnbocd/mergiraf_a_driver_for_git_merge_that_uses_an/